

The background features a large, stylized grey letter 'U' that serves as a container for the title. Above the 'U', there is a white rectangular area containing four red circles of varying sizes and a red horizontal bar. The top of the page is decorated with a red header bar that has a wavy, layered appearance.

DEFINICIÓN Y EJECUCIÓN DE UN ENDPOINT SENCILLO

TFG

Diego Carrera Hernando
Grado en Ingeniería Informática

Contenido

Introducción	2
Implementación y ejecución del endpoint.....	2
Dependencias	2
ExampleEndpoint	2
Imports	2
Definición del endpoint.....	3
Lógica del endpoint	4
Rutas HTTP4	4
Main	5
Imports	5
Método “Run”	5
Ejecución	6

Introducción

En este PDF explicaré detalladamente cómo especificar un endpoint con la librería Tapir y cómo ejecutarlo con el framework http4s. Para ello, mostraré, paso a paso, un endpoint sencillo que voy a crear para ilustrar el procedimiento seguido.

Implementación y ejecución del endpoint

Con tal fin, he decidido que la funcionalidad del endpoint consista en devolver los identificadores de los elementos favoritos (una película, una serie, un videojuego y un libro) de un hipotético usuario. Es decir, este endpoint proporcionará cuatro identificadores (números del tipo “long”) como respuesta a una solicitud GET cuyo parámetro de ruta sea el nombre de usuario.

Para ello, dividiremos el código en dos ficheros diferentes. Un fichero será “ExampleEndpoint”, que contendrá una clase homónima en la que se definirá el endpoint de ejemplo, así como la lógica de implementación y las rutas HTTP. El otro fichero será “Main”, que contiene el código ejecutado por la aplicación.

Dependencias

Para poder llevar a cabo la definición y ejecución del endpoint deseado, lo primero de todo es incluir ciertas dependencias en el proyecto. Las dependencias en cuestión son las siguientes:

```
"com.softwaremill.sttp.tapir" %% "tapir-core" % "1.9.11",  
"com.softwaremill.sttp.tapir" %% "tapir-json-circe" % "1.9.11",  
"com.softwaremill.sttp.tapir" %% "tapir-http4s-server" % "1.9.11",  
"org.http4s" %% "http4s-blaze-server" % "0.23.16",  
"ch.qos.logback" % "logback-classic" % "1.4.14"
```

Las tres primeras dependencias nos permitirán importar las clases fundamentales para crear descripciones de endpoints, usar la librería Circe en el proyecto para trabajar con objetos JSON, y crear servidores http4s, respectivamente.

En cuanto a las otras dependencias, “http4s-blaze-server” nos permite utilizar el backend de http4s para el servidor, mientras que “logback-classic”, al usar “BlazeServerBuilder” de “http4s-blaze-server” (se verá más adelante) me saltaba un error que no alcanzaba a comprender demasiado bien, pero, por lo que he entendido, esta dependencia utiliza la librería SLF4J para el registro de eventos, por lo que, si se desea utilizar, se debe agregar “logback-classic” al resto de dependencias mencionadas.

ExampleEndpoint

Este fichero, como ya he comentado antes, contendrá la definición del endpoint propuesto, la lógica de implementación y las rutas HTTP.

Imports

Para poder usar clases, métodos o tipos necesarios necesarios a la hora de definir el endpoint, debemos realizar algunos imports, tal y como se muestra a continuación:

```
import sttp.tapir._
import sttp.tapir.json.circe._
import sttp.tapir.server.http4s.Http4sServerInterpreter

import cats.effect.IO

import org.http4s.HttpRoutes
```

Los tres primeros imports sirven, respectivamente, para acceder a gran parte de las funcionalidades de Tapir, usar la librería Circe para trabajar con objetos JSON y crear nuestro propio servidor http4s.

Los dos siguientes imports sirven para poder usar la clase “IO” de Cats Effect y para poder crear rutas HTTP con http4s.

Definición del endpoint

En cuanto a la definición del endpoint, lo primero que hay que tener en cuenta es que, al menos en este endpoint concreto, no vamos a utilizar parámetros de entrada de seguridad, por lo que podemos definir el tipo “PublicEndpoint”, que será igual que el tipo “Endpoint” de Tapir, pero sin parámetros de entrada de seguridad. De este modo, “PublicEndpoint” tendrá parámetros de entrada, de salida de error, de salida, y las características requeridas por dichos parámetros.

```
private type PublicEndpoint[I, E, O, -R] = Endpoint[Unit, I, E, O, R]
```

Definiremos también un endpoint base del que partirán el resto de endpoints relacionados con el usuario (como el que estamos definiendo):

```
private val userBaseEndpoint: PublicEndpoint[Unit, Unit, Unit, Any] =
  endpoint.in("api" / "user")
```

Lo siguiente que podemos hacer, aunque no sea estrictamente necesario, es definir las entradas y salidas, que podremos utilizar para distintos endpoints, por lo que puede constituir una buena práctica definirlos. Un ejemplo de esto sería crear un valor del tipo EndpointInput que se corresponda con un parámetro de ruta de la siguiente manera:

```
private val pathUserId: EndpointInput[Long] =
  path[Long](name = "user_id")
```

O, por ejemplo, también podemos crear un valor del tipo EndpointOutput para establecer el valor que se devuelve. En este caso, lo que devuelve es un objeto JSON con la lista de identificadores que constituyen la sección “Favoritos” del perfil de un usuario concreto.

```
private val jsonLongListOut: EndpointOutput[Seq[Long]] =
  jsonBody[Seq[Long]]
```

Ahora lo que haremos será utilizar todos los pequeños endpoints definidos anteriormente para crear el endpoint que queríamos crear inicialmente. Por tanto, la ruta base del endpoint será la definida en `baseEndpoint`, la entrada será la concatenación del parámetro de ruta definido en `EndpointInput` con la cadena “favourites”, y la salida será la definida en `EndpointOutput`.

```
val favouritesEndpoint: PublicEndpoint[Long, String, Seq[Long], Any] =  
  userBaseEndpoint  
    .in(pathUserId)  
    .in("favourites")  
    .out(jsonLongListOut)  
    .errorOut(stringBody)
```

Por otra parte, para añadir más información al endpoint definido, también podemos usar los métodos “name()”, “description()” y “get”, que sirven para establecer en la documentación el nombre, la descripción y la solicitud HTTP del endpoint, respectivamente.

```
val favouritesEndpoint: PublicEndpoint[Long, String, Seq[Long], Any] =  
  userBaseEndpoint  
    .name("User Favorites endpoint")  
    .description("This endpoint returns the favorite elements IDs of the specified user")  
    .get  
    .in(pathUserId)  
    .in("favourites")  
    .out(jsonLongListOut)  
    .errorOut(stringBody)
```

Aun así, en este ejemplo aún no vamos a generar la documentación del endpoint en OpenAPI o Swagger, por lo que no hace falta incluir estos métodos de momento.

Lógica del endpoint

Lo siguiente que vamos a hacer es implementar la lógica del endpoint, para lo cual vamos a crear una función que recibe un identificador de usuario de tipo “Long” y devuelva una lista con los identificadores de los elementos favoritos del usuario, o bien un mensaje de error. Como esto no es más que un ejemplo, el código está hecho únicamente para devolver una lista de valores cuando se le pasa un identificador de usuario concreto.

```
val favouritesEndpointLogic: Long => IO[Either[String, Seq[Long]]] = id =>  
  if (id == 1) IO.pure(Right(List(502033, 61222, 113112, 12354)))  
  else IO.pure(Left("Item not found"))
```

Rutas HTTP4

Para finalizar todo lo referente a la clase “ExampleEndpoint”, procedemos a definir las rutas HTTP a partir del endpoint y de la lógica de implementación de este, descritos anteriormente.

```
val returnFavouritesRoutes: HttpRoutes[IO] =
  Http4sServerInterpreter[IO]().toRoutes(favouritesEndpoint.serverLogic(favouritesEndpointLogic))
```

Main

Habiendo hecho ya todo lo anterior, procedemos a implementar el código que se encarga de la ejecución del endpoint. Para ello, nos situamos en un fichero aparte, llamado “Main”.

Imports

Antes de nada, debemos realizar algunos imports, entre los que también se encuentra el de la clase anterior en la que se define el endpoint, tal y como se muestra a continuación:

```
import cats.effect.{IO, ExitCode, IOApp}
import org.http4s.blaze.server.BlazeServerBuilder
import org.http4s.implicits._
import api.ExampleEndpoint
```

Estos imports sirven para acceder funcionalidades de Cats Effect, así como para crear nuestro propio servidor http4s con el backend Blaze.

Método “Run”

Este método es el encargado de la ejecución de la aplicación y, antes de nada, cabe recalcar que el objeto Main en el que se encuentra extiende el trait “IOApp”.

```
Diego *
object Main extends IOApp {
```

Este trait proporciona un punto de entrada para las aplicaciones que están escritas usando el tipo “IO” de Cats Effect, que permite el manejo integrado de efectos secundarios y errores, y control de finalización limpio para la aplicación.

En cuanto al método “Run”, su código es el que se muestra a continuación:

```
Diego *
override def run(args: List[String]): IO[ExitCode] = {
  val endpoint = new ExampleEndpoint()

  val routes = endpoint.returnFavouritesRoutes

  BlazeServerBuilder[IO]
    .bindHttp(port = 8080, host = "localhost")
    .withHttpApp(routes.orNotFound)
    .serve
    .compile
    .drain
    .as(ExitCode.Success)
}
```

Como podemos observar, lo primero que hacemos en este método es crear una instancia de la clase “ExampleEndpoint” y asignar el valor devuelto por el método “returnFavouritesRoutes” a una constante. Después, utilizamos “BlazeserverBuilder[IO]” para construir un servidor http4s, siendo “IO” el efecto base de este, y configuramos el servidor para que escuche en el puerto 8080 y en la interfaz de red local, por lo que estará disponible en `http://localhost:8080`. Tras ello, especificamos la aplicación HTTP a ejecutar en nuestro servidor con la ruta del endpoint definido y el método “orNotFound”, que maneja las solicitudes no coincidentes con la ruta definida. A continuación, iniciamos el servidor con el método “serve”, compilamos el efecto “IO” con el método “compile”, y con “drain” se ignora el resultado de dicho efecto, pues únicamente queremos ejecutarlo. Por último, convertimos el efecto “IO” en un código que nos indicará que la aplicación se ha ejecutado correctamente. Dicho código se puede ver a continuación

Ejecución

Por último, nos queda probar que el código funciona y que lo hace de la manera deseada. Para ello, ejecutamos el programa y, desde la línea de comandos de Windows, lanzamos una petición GET a la dirección URL que establece el endpoint. Es decir, deberíamos escribir el siguiente comando:

```
C:\Users\diego>curl -X GET http://localhost:8080/api/user/1/favourites
```

Al haber hecho una solicitud a la dirección propuesta en el endpoint y con el parámetro de ruta (“user_id”) admitido por la lógica establecida, la aplicación nos devolverá la lista con los cuatro elementos de tipo “Long”, tal y como se muestra aquí:

```
[502033,61222,113112,12354]
```

En el caso contrario, es decir, cuando el parámetro de ruta toma un valor no manejado por la lógica del endpoint, se nos devolverá el siguiente mensaje:

```
Item not found
```

Por otra parte, si en la petición se introduce una URL que no se corresponda con la definida en el endpoint, se devolverá el mensaje mostrado a continuación:

```
Not found
```